

## 5.2 AI-Supported Test Automation

If you work on a new feature for your application, there will come a time when you need to check the functionality. A manual check is time consuming and prone to errors, especially if you consider that you also have to test at least parts of the rest of the application, as errors may have crept in at other points as a result of the integration. The automation potential for tasks such as checking the features of an application is huge. This is precisely why the discipline of test automation exists. You can test your application at various levels based on the *test pyramid*, which classifies the different types of tests at different levels:

- **Unit tests**

The basis of the pyramid is formed by the unit tests. They check the smallest testable units of an application in isolation. These units are usually functions or methods. Unit tests are processed quickly, are easy to write, and are usually available in large numbers.

- **Integration tests**

These tests form the middle level of the test pyramid. They test individual or several modules or components. Integration tests are more complex and slower than unit tests, but they also provide a higher degree of security as they test the interaction of multiple components in your application. Typically, your application has significantly fewer integration tests than unit tests.

- **End-to-end tests**

End-to-end tests (sometimes referred to as E2E tests) form the top of the pyramid. They test features across the entire application, from the frontend to data management. These tests are the most complex to create and the slowest to run, but they provide the highest level of security. Compared to the other two levels, end-to-end tests typically take up less room.

AI tools can support you at all levels of the test pyramid and relieve you of some of the work involved in generating tests. In the remaining sections of this chapter, you'll learn about the different levels of the test pyramid and their characteristics using TypeScript as an example. However, tests work similarly in other programming languages and environments, so you can transfer your knowledge.

### 5.2.1 Creating AI-Supported Unit Tests

There are multiple testing frameworks for unit tests in JavaScript and TypeScript. One of the most popular is Vitest, a testing framework that comes from the Vue.js ecosystem, but is framework- and even platform-independent. That is why you can use the testing framework both on the client side in the browser and on the server side using Node.js. A major advantage of Vitest is that it doesn't require any configuration at all in simple cases. It automatically finds the tests based on their file name and executes them.

#### AI-Supported Unit Tests

The first step in the AI-supported generation of a unit test is a simple TypeScript function named `add`, which adds two integers and returns the result. The code for this function looks as follows:

```
export default function add(a: number, b: number):
number {
  if (!Number.isSafeInteger(a) ||
!Number.isSafeInteger(b)) {
    throw new Error("Can only add integers");
  }
  return a + b;
}
```

You can have ChatGPT generate a test for this function:

**Prompt:** Please write a Vitest unit test for the following function:

[code of the function]

Create only the test; the environment is already set up, and everything is installed.

```
import { describe, it, expect } from 'vitest';
import add from './add'; // Adjust path to function
add

describe('add', () => {
  it('should add two integers correctly', () => {
    expect(add(2, 3)).toBe(5);
    expect(add(-1, 1)).toBe(0);
    expect(add(0, 0)).toBe(0);
  });
  it(
    'should throw an error if either argument is not a  
safe ' +
    'integer', () => {
      expect(() => add(2.5, 3))
        .toThrow('Can only add integers');
      expect(() => add(2, 3.5))
        .toThrow('Can only add integers');
      expect(() => add(Number.MAX_SAFE_INTEGER + 1, 3))
        .toThrow('Can only add integers');
      expect(() => add(2, Number.MAX_SAFE_INTEGER + 1))
        .toThrow('Can only add integers');
    });
  it(
    'should throw an error if any argument is not a  
number',
    () => {
      expect(() => add('2', 3))
        .toThrow('Can only add integers');
      expect(() => add(2, '3'))
        .toThrow('Can only add integers');
      expect(() => add(null, 3))
        .toThrow('Can only add integers');
      expect(() => add(2, undefined))
        .toThrow('Can only add integers');
    });
});
```

[explanation of the scenarios]

If you specify the entire source code in the prompt, as in this example, the AI tool can cover all aspects of the functionality. The signature of the function contributes to this because here you specify exactly which data types you expect, and which type you return. The error handling within the function indicates additional limit and exception cases that must also be covered by the tests. AI tools such as ChatGPT like to be a little more detailed in their answers to such tasks and not only generate the requested test but also provide tips on setup and execution. You can prevent this by specifying that you're only interested in the test and not in the environment or the installation.

The result produced by ChatGPT is impressive. Not only does the generated code contain a simple test that adds the values 2 and 3, for example, and checks whether the correct result is returned, it contains a whole series of tests. What stands out positively here is that the tests are grouped using `describe`, and the group has been given the name of the function as a label. There are three tests within this group:

- **Successful case**

The test expects the function to be able to add two integers. The first test ensures that two positive integers can be added, the second one adds a negative number, and the third test ensures that 0 and 0 can also be added.

- **Error case 1**

The predefined implementation of the `add` function checks that both numbers are *safe integers*, that is, integers that lie in a range in which TypeScript or the JavaScript engine can calculate them correctly. The second test checks that calculating with floating point numbers and numbers that are too large (`Number.MAX_SAFE_INTEGER + 1`) isn't possible and leads to an exception.

- **Error case 2**

The third test goes even further and ensures that calling the `add` function with invalid data types such as strings, `null`, or `undefined` leads to an error.

If successful, ChatGPT uses the `toBe` matcher to check whether the result matches the expected value. The choice of this check function fits well, as it checks internally for strict equality and doesn't allow type conversion. This function only causes difficulties if reference types such as objects are involved, which isn't the case here. The error cases correctly cover the tests using the `toThrow` matcher, which you can use to check whether a function throws an error.

Let's now move on to the weaknesses of the generated tests. A major problem is that the TypeScript compiler reports some errors in the tests when processing the code. The third test, which calls the function with different types, isn't possible in TypeScript because the types aren't compatible with the signature of the function when it's called. The prompt doesn't mention anything about TypeScript and type-safe code, but this should have been obvious when the function was analyzed.

You can address this problem by requiring ChatGPT to produce valid TypeScript code. To do this, you can either adapt the original prompt or specify the request as shown here:

**Prompt:** Make sure the tests are written in valid TypeScript. Keep the test if any argument isn't a number, but fix it.

Sure, here is the corrected test suite in valid TypeScript:

```
(existing code ...)
it(
  'should throw an error if any argument is not a
  number',
  () => {
    expect(() => add('2' as any, 3))
      .toThrow('Can only add integers');
    expect(() => add(2, '3' as any))
      .toThrow('Can only add integers');
    expect(() => add(null as any, 3))
      .toThrow('Can only add integers');
    expect(() => add(2, undefined as any))
      .toThrow('Can only add integers');
```

```
});  
});
```

In this version: each invalid argument is explicitly cast to any to ensure the test case runs as expected while maintaining TypeScript type checks for valid arguments.

To be on the safe side, you should specify in the prompt that the third test should be retained in any case. This is unnecessary in a pure TypeScript environment, as you can't execute the function with an incorrect type. However, because TypeScript is only used at compile time and not at runtime, the wrong data types may still be used depending on the environment. The original two tests remain unchanged, as they were already written in valid TypeScript. To avoid the errors in the third test, ChatGPT uses TypeScript's type conversion and turns an invalid string data type into a number. Although this adaptation isn't elegant, it satisfies the TypeScript compiler.

Another point of criticism is that the tests don't just have one assertion, but always more than one. Although the tests always fit together thematically, there is often a requirement that each test should only have one assertion. The reason for this is that a test should only fail for one reason in order to facilitate troubleshooting. In this case, it's up to you to decide whether you would prefer to have fewer, thematically grouped tests, or many, finely granular tests. You can also solve this issue by extending the prompt and asking ChatGPT to create a separate test for each expectation:

**Prompt:** Please put every expectation in its own test. And keep the grouping by putting the tests in their own sub-describe blocks.

Sure, here is the revised test suite with each expectation in its own test and grouped using sub-describe blocks:

```
import { describe, it, expect } from 'vitest';  
import add from './add'; // Adjust path to function  
add  
  
describe('add', () => {
```

```
describe('valid inputs', () => {
  it('should add 2 and 3 correctly', () => {
    expect(add(2, 3)).toBe(5);
  });

  ... more tests ...
});

describe('invalid safe integer inputs', () => {
  ... more tests ...
});

describe('invalid non-number inputs', () => {
  ... more tests...
});
});
```

[explanation of the code]

If you only prompt the first sentence, ChatGPT turns the original 3 tests into a total of 11 tests. The second sentence of the prompt ensures that the grouping gets maintained and that you don't lose the overview so quickly.

In general, you'll have to deal with the topic of testing, although tools such as ChatGPT, Copilot, and the like can do a lot of work for you. You should familiarize yourself with the structure and organization of tests and develop a basic knowledge of test patterns and the architecture of tests. The more precisely you formulate your prompts, the more likely the result will meet your expectations, and the code of the tests will remain readable and maintainable in the long term.

However, the possibilities of AI don't end with the formulation of tests; the tools can also support you in test-driven development.